You've added the concept of roles to handle the list of users attached to a project. The test suggests that project has a relationship to roles—in fact, the first point of failure for the test will be that call to project.roles.

You could create a whole round of integration testing to drive the addition of a user interface for adding users to projects and allow the data model to be driven as part of that process. Or you could hand-wave the UI, add a basic data model, and focus on the security aspect.

In a full project the add-user-to-project story would get full treatment. For our purposes, you can start with just a migration and a new model. We'll assume somebody else did the add-user-to-project story.

Let's create the migration:

```
$ rails generate model role user:references project:references role_name:string
$ rake db:migrate
```

That gives you the following migration:

security/02/db/migrate/20170904004010_create_roles.rb

```
class CreateRoles < ActiveRecord::Migration[5.1]
  def change
    create_table :roles do |t|
      t.references :user, foreign_key: true
      t.references :project, foreign_key: true
      t.string :role_name

      t.timestamps
    end
  end
end
```

This goes a tiny bit beyond the test by adding a role_name.

You'll also need to add the associations—first to Project:

security/02/app/models/project.rb

```
has_many :roles, dependent: :destroy
has_many :users, through: :roles
```

and then to User:

security/02/app/models/user.rb

```
has_many :roles, dependent: :destroy
has_many :projects, through: :roles
```

At this point, the test has the expected failure, which is that you want to block access to a user who's not part of the project, but the code doesn't have that block yet.